Python Basic Assignment — Solutions

Solutions to all 20 questions. The grade thresholds in Control 3 are explicit assumptions because the assignment does not specify a grading scale.

1. List comprehension + condition

Python / SQL code

```
squares = [n**2 for n in range(1, 21) if n % 2 == 0]
print(squares)
```

Sample output

```
[4, 16, 36, 64, 100, 144, 196, 256, 324, 400]
```

2. Nested list access

Python / SQL code

```
data = [[1, 2], [3, 4], [5, [6, 7]]]
print(data[2][1][1])
```

Sample output

```
7
```

3. Remove duplicates and sort

Python / SQL code

```
def unique_sorted(numbers):
    return sorted(set(numbers))

print(unique_sorted([4, 2, 4, 1, 3, 2]))
```

Sample output

```
[1, 2, 3, 4]
```

4. Tuple unpacking

Python / SQL code

```
data = [(1, 2), (3, 4), (5, 6)]
for a, b in data:
    print(a + b)
```

Sample output

```
3
7
11
```

5. Immutable check

Tuple elements cannot be reassigned. The attempted assignment raises TypeError.

Python / SQL code

```
point = (10, 20)
try:
    point[0] = 99
except TypeError as error:
    print(type(error).__name__ + ":", error)
```

Sample output

```
TypeError: 'tuple' object does not support item assignment
```

6. Character frequency count

Python / SQL code

```
from collections import Counter
def character_count(text):
    return dict(Counter(text))

print(character_count("hello"))
```

Sample output

```
{'h': 1, 'e': 1, 'l': 2, 'o': 1}
```

7. Merge dictionaries; add duplicate-key values

Python / SQL code

```
d1 = {"a": 10, "b": 20}
d2 = {"b": 5, "c": 30}
merged = d1.copy()
for key, value in d2.items():
    merged[key] = merged.get(key, 0) + value
print(merged)
```

Sample output

```
{'a': 10, 'b': 25, 'c': 30}
```

8. Common elements using sets

Python / SQL code

```
def common_elements(a, b):
    return sorted(set(a) & set(b))

print(common_elements([1, 2, 3, 4], [3, 4, 5]))
```

Sample output

```
[3, 4]
```

9. Symmetric difference

Python / SQL code

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
print(sorted(a ^ b))
```

Sample output

```
[1, 2, 5, 6]
```

10. Group words by length

Python / SQL code

```
from collections import defaultdict
words = ["cat", "dog", "apple", "bat"]
groups = defaultdict(list)
for word in words:
    groups[len(word)].append(word)
print(dict(groups))
```

Sample output

```
{3: ['cat', 'dog', 'bat'], 5: ['apple']}
```

Control 1. If-else: even or odd

Python / SQL code

```
number = 8
if number % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Sample output

```
Even
```

Control 2. Nested if: positive, negative or zero

Python / SQL code

```
number = -3
if number >= 0:
    if number == 0:
        print("Zero")
    else:
        print("Positive")
else:
    print("Negative")
```

Sample output

```
Negative
```

Control 3. If-elif-else: grade

Assumed grading scale: A >= 90, B >= 75, C >= 50, otherwise Fail. Change thresholds if your institution uses another scale.

Python / SQL code

```
marks = 82
if marks >= 90:
    grade = "A"
elif marks >= 75:
    grade = "B"
elif marks >= 50:
    grade = "C"
else:
    grade = "Fail"
print(grade)
```

Sample output

```
B
```

Control 4. Multiplication table using for loop

Python / SQL code

```
number = 5
for i in range(1, 11):
    print(f"{number} x {i} = {number * i}")
```

Sample output

```
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
```

Control 5. While loop: 10 down to 1

Python / SQL code

```
n = 10
while n >= 1:
    print(n)
    n -= 1
```

Sample output

```
10
9
8
7
6
5
4
3
2
1
```

Control 6. Break when user enters 0

Python / SQL code

```
while True:
    number = int(input("Enter a number (0 to stop): "))
    if number == 0:
        print("Stopped")
        break
    print("You entered:", number)
```

Sample output

```
Example interaction:
Enter a number (0 to stop): 4
You entered: 4
Enter a number (0 to stop): 0
```

```
Stopped
```

Control 7. Continue: print even numbers

Python / SQL code

```
for n in range(1, 11):
    if n % 2 != 0:
        continue
    print(n)
```

Sample output

```
2
4
6
8
10
```

Control 8. Pass statement

Python / SQL code

```
def future_function():
    pass

print("Function defined successfully")
```

Sample output

```
Function defined successfully
```

Control 9. Prime check using for-else

Python / SQL code

```
number = 17
if number < 2:
    print("Not prime")
else:
    for divisor in range(2, int(number**0.5) + 1):
        if number % divisor == 0:
            print("Not prime")
            break
    else:
        print("Prime")
```

Sample output

```
Prime
```

Control 10. Pattern using nested loops

Python / SQL code

```
for row in range(1, 5):
    for col in range(row):
        print("*", end="")
    print()
```

Sample output

```
*
**
***
****
```